KUL

CHANGE REPORT

Parallel abc_new in Yosys

abc_new maps CVA6 identically with up to 1.47 times less wall time

AUTHOR	rafay
DATE	2026-10-24
STATUS	Draft, pushed, not merged
DESIGN AND PLATFORM	cva6 on asap7, hierarchical
KUL BRANCH	abc-new-parallel at 1a7fd02
REVIEWERS	Not yet assigned

Contents

1. Summary 3

2. Change at a glance 3

3. Background 3

3.1 What ABC does 4

3.2 What abc_new does 4

3.3 Why abc_new ran on one core 5

4. Design 5

4.1 How it works 5

4.2 Ordering, determinism and safety 6

4.3 Options 6

5. Code and config changes 6

5.1 Commits 6

5.2 Usage 7

6. Verification 7

6.1 Regression tests 7

6.2 A/B method 7

6.3 Results 8

7. Results 8

7.1 Quality of results 8

7.2 Runtime 8

7.3 Where the time goes 9

8. Issues found and fixes 9

8.1 Box-id assertion (crash) 9

8.2 Leftover $input_port cells 10

9. Repositories and publishing 10

9.1 Published 10

9.2 To reproduce 11

10. Limitations and next steps 11

10.1 Limitations 11

10.2 Next steps 11

11. Future work 11

11.1 Per-module buffer normalization 11

1. Summary

Yosys abc_new now runs ABC on independent modules in parallel. On CVA6 (asap7, hierarchical) it maps all 93 modules to exactly the same cells as the sequential version, in up to 1.47 times less wall time.

- **Change:** abc9_exe splits into a prepare step and a collect step; abc_new runs the external yosys-abc processes in between on a thread pool, behind a new `-j <num>` option.
- **Correctness:** 818,111 mapped cells match the sequential run by module and cell type. -j 4 and -j 8 give byte-identical output. All 44 abc_new regression tests pass in both modes.
- **QoR and runtime:** cells unchanged; abc_new wall time 2,052 s sequential, 1,398 to 1,730 s with -j 8. The cap is about 740 s of serial whole-design buffer normalization in write_xaiger2 and portarcs.
- **Where it lives:** six commits on Pr0Chef/yosys branch abc-new-parallel (head 7b7e811cb), pinned as the yosys submodule on Pr0Chef/KUL branch abc-new-parallel (1a7fd02). Pushed, not merged.

2. Change at a glance

Field	Value
Owner	rafay
Status	Draft, pushed to Pr0Chef forks, not merged
KUL branch	abc-new-parallel at 1a7fd02
Tool branches	Pr0Chef/yosys abc-new-parallel at 7b7e811cb
Yosys version	Yosys 0.69+274 (base 7513f8df8) plus this branch
OpenROAD version	26Q4-271-g0907a8656e
ORFS	vendored orfs/ in KUL
Design / platform	cva6 / asap7
Config and variant	designs/asap7/cva6/config_hier.mk, FLOW_VARIANT=hier_par
Stages affected	synth (1_2_yosys, the abc_new step)
Machine	12th Gen Intel Core i5-1235U, 8 threads, 10 GB RAM

3. Background

CVA6 hierarchical synthesis spent 1,434 s of its 1 h 58 min Yosys step in abc_new, on a single core. The classic abc pass already runs ABC in parallel, but the CVA6 flow never reaches it.

- **Flow setting:** config_hier.mk sets SYNTH_HIERARCHICAL = 1 and SYNTH_MINIMUM_KEEP_SIZE = 10000, keeping CVA6 as 93 modules for ABC.

- **Code path:** config.mk sets SWAP_ARITH_OPERATORS = 1, so synth.tcl calls abc_new instead of abc, letting each arithmetic operator module use its own ABC script.
- **Existing behaviour:** classic abc runs one yosys-abc process per module on a thread pool (abc.cc). abc_new was a plain loop: write box, write xaiger, run ABC, read result, reintegrate, one module after another.
- **Evidence:** run_logs/cva6_asap7_hier_synth_20261019_161727.log: "Time spent: 17% 1x abc_new (1434 sec), 13% 1x extract_fa (1048 sec)".

3.1 What ABC does

ABC is a logic synthesis and verification tool from UC Berkeley. Yosys uses it for the last step of synthesis: turning generic logic into cells of the target standard-cell library, asap7 here.

- **Input:** Yosys writes the combinational logic of a module as an and-inverter graph (AIG): only 2-input AND gates and inverters, with flip-flops and black boxes as cut points.
- **Optimization:** ABC restructures the graph to make it smaller and shallower. The ORFS script uses commands such as `&st`, `&dch`, `&syn2` and `&synch2`.
- **Technology mapping:** ABC covers the graph with library cells read from the Liberty files, choosing them for delay against the constraint file (`&nf`).
- **Output:** a mapped netlist that Yosys stitches back into the module in place of the generic logic.

ABC runs as a separate program, yosys-abc, and each run uses a single core.

3.2 What abc_new does

abc_new is the newer of Yosys's two ABC flows and is still marked experimental. Like the classic abc pass it maps the design one module at a time, but it gives ABC more context:

- **Boxes with timing:** submodules marked abc9_box, here the wrapped arithmetic operators, appear to ABC as boxes with delay arcs, so mapping a parent module accounts for how slow its adders are.
- **Per-module scripts:** a module can carry its own ABC script in its abc9_script attribute. ORFS gives operator modules a shorter one, `+&dch;&nf -R 5;`.
- **Why ORFS uses it for CVA6:** SWAP_ARITH_OPERATORS = 1 makes Yosys build each adder in several architectures (Han-Carlson, Kogge-Stone, Sklansky, Brent-Kung) and multipliers as Booth variants, each as its own module. OpenROAD's replace_arith_modules can then swap in a better architecture during floorplan and resizing. Those operator modules need abc_new's boxes and per-module scripts.

For each module, abc_new runs five Yosys commands in order:

1. `abc9_ops -write_box`: write the box library, the delay arcs of every box module.
2. `write_xaiger2`: write the module as an XAIGER file, an AIG with its boxes.
3. `abc9_exe`: run yosys-abc on that file and wait for it to finish.
4. `read_aiger`: read the mapped result back into Yosys.
5. `abc_ops_reintegrate`: replace the module's generic logic with the mapped cells.

If the module is itself a box, an arithmetic operator, portarcs then measures its delay arcs and abc9_ops -prep_box registers it, so the modules that use it see its timing.

3.3 Why abc_new ran on one core

abc_new is a script pass: a fixed list of Yosys commands that it runs one after another, in a plain for loop over the modules.

- **One ABC run at a time:** each command waits for the previous one, and abc9_exe blocks until its ABC process exits. With 93 CVA6 modules there was never more than one ABC process running.
- **Shared state:** the commands read and change the shared design database and write to the shared log. Neither is thread-safe in Yosys, so the commands cannot simply be run side by side.
- **Real ordering:** an operator module must be mapped and its timing measured before any module that instantiates it is written out for ABC.
- **Not done before:** the classic abc pass already runs its ABC processes on a thread pool (abc.cc). abc_new, still experimental, had not had the same work.

The change in section 4 keeps all three rules: only the ABC process runs on a worker thread, only the main thread touches the design and the log, and a module starts only after its submodules are mapped.

4. Design

Only the ABC processes leave the main thread. Everything that touches the design stays on the main thread, in an order that depends only on the module order.

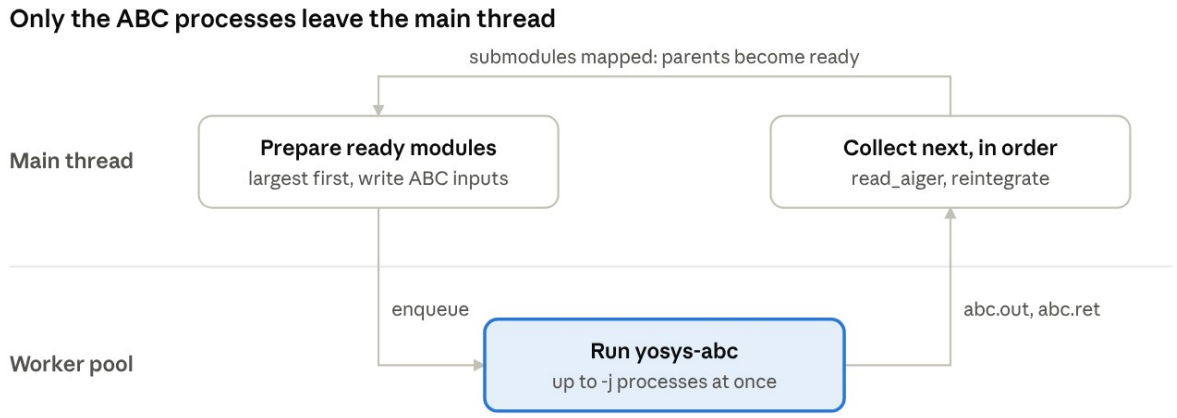


Figure 1. abc_new parallel path: two threads of control, three steps per module.

4.1 How it works

1. **Prepare (main thread):** make a temp dir, apply the module's abc9_script override, run abc9_ops -write_box, write_xaiger2 -mapping_prep and abc9_exe -prepare; record the ABC command and a snapshot of every abc9_box_id.
2. **Run (worker thread):** std::system("<cmd> > <tmp>/abc.out 2>&1"), then write the exit status to <tmp>/abc.ret. Workers never touch the design or the log.
3. **Collect (main thread):** abc9_exe -collect logs abc.out and checks the status; read_aiger and abc_ops_reintegrate run under the snapshotted box ids; a deferred box then gets portarcs and abc9_ops -prep_box.

4.2 Ordering, determinism and safety

- **Ordering:** a module starts once all its selected submodules are collected, the same edges order_modules() sorts by, so parents see their deferred boxes' timing. Ready modules are enqueued largest first.
- **Determinism:** readiness depends only on how many modules are collected, never on timing, so the main thread's design edits (and autoidx names) are the same for every -j above 1. -j 4 and -j 8 were byte-identical on CVA6.
- **Fallbacks:** -j 1 keeps the original loop. Builds without YOSYS_ENABLE_THREADS, with YOSYS_LINK_ABC, or for wasm or Windows always use it. A QueueCloser lets workers exit cleanly when an error unwinds.

4.3 Options

Option or variable	Default	Effect
abc_new -j <num>	number of cores	Maximum concurrent ABC processes; 1 selects the original sequential loop
YOSYS_MAX_THREADS	unset	Caps the default pool size

5. Code and config changes

2 files changed, +292 / −9 lines (git diff --stat 7513f8df8 7b7e811cb). No ORFS script or config changed.

File	Change
passes/techmap/abc9_exe.cc	New Abc9ExeMode (RUN, PREPARE, COLLECT) and options -prepare / -collect. Prepare writes abc.script and stores the command in the abc9_exe.command scratchpad variable; collect logs <cwd>/abc.out and checks <cwd>/abc.ret. Return-code check shared in check_abc9_result().
passes/techmap/abc_new.cc	New -j option, AbcJob and run_abc_job() worker, map_parallel() scheduler with prepare_module() and collect_module(), box-id snapshot and restore, removal of leftover $input_port cells.

5.1 Commits

Repository	Branch	Commit	What
KUL	abc-new-parallel	1a7fd02	Add yosys as a submodule with parallel abc_new
KUL	abc-new-parallel	2c7a820	Stop ignoring the yosys directory
yosys	abc-new-parallel	7b7e811cb	abc_new: start the largest ready modules first
yosys	abc-new-parallel	db1edb158	abc_new: remove $input_port cells left after

Repository	Branch	Commit	What
			parallel mapping
yosys	abc-new-parallel	48001b2d4	abc_new: keep box ids stable between prepare and collect
yosys	abc-new-parallel	c32e5780c	abc_new: map independent modules in parallel
yosys	abc-new-parallel	0ea6ecd87	abc9_exe: add -prepare and -collect modes
yosys	abc-new-parallel	db250e0ca	abc9_exe: share the ABC exit status check

Newest first. Each Yosys commit builds on its own; the last one passes all 44 abc_new tests in both modes.

5.2 Usage

```
abc_new -j 8 -liberty ... -constr ...   # up to 8 concurrent ABC processes
abc_new -j 1 ...                        # original sequential loop
YOSYS_MAX_THREADS=4 yosys ...           # caps the default pool size
```

ORFS calls abc_new without -j, so CVA6 synthesis takes the parallel path with no flow change.

6. Verification

All 44 abc_new regression tests pass in both modes, and CVA6 maps identically to the sequential loop.

6.1 Regression tests

Suite	Tests	Mode	Result
yosys tests/abc_new	30	parallel and sequential	30 / 30 pass
tests/abc_new/librelane	6	parallel and sequential	6 / 6 pass
tests/abc_new/openroad	8	parallel and sequential	8 / 8 pass

Eight of these have more than one selected module and reach the parallel path: bug6179, hier_check, paramod_equiv, whitebox_equiv, whitebox_no_timing, read_liberty_lib, spm_hier_equiv and cm_cases.

6.2 A/B method

1. **Baseline:** full ORFS hierarchical synthesis with the new Yosys, FLOW_VARIANT=hier_par.
2. **Checkpoint:** a patched copy of synth.tcl wrote pre_abc.rtlil (80 MB) and the abc_new arguments just before abc_new.

3. **Replay:** abc_new alone on the checkpoint with -j 8, -j 4 and -j 1, from orfs/flow (the constraint file path is relative).
4. **Compare:** stat per module and cell type, excluding $input_port bookkeeping cells and the hierarchy summary.

6.3 Results

- **Equivalence:** identical: 1,999 module and cell-type rows match, 818,111 cells. A sample ALU module shows the same ABC input (394 ANDs, 64 boxes) and output (683 ANDs, 16 levels) in both modes.
- **Determinism:** -j 8 and -j 4 outputs are byte-identical (130,339,186 bytes each).
- **Not yet done:** a full ORFS synthesis end to end with the final build, a gate-level equivalence check of the whole design, and a regression test for the box-id case.

7. Results

-j 8 takes abc_new on CVA6 from 2,052 s to between 1,398 and 1,730 s, with the same mapped cells and 21% lower peak memory. Run-to-run spread is as large as the gain from the scheduling tweak.

7.1 Quality of results

Stage	Metric	Baseline (-j 1)	With change (-j 8)	Delta
Synth	Mapped cells (excluding $input_port)	818,111	818,111	0
Synth	Leftover $input_port cells	1,300	0	−1,300

Later stages were not run with the final build; the netlist they would receive has the same cells.

7.2 Runtime

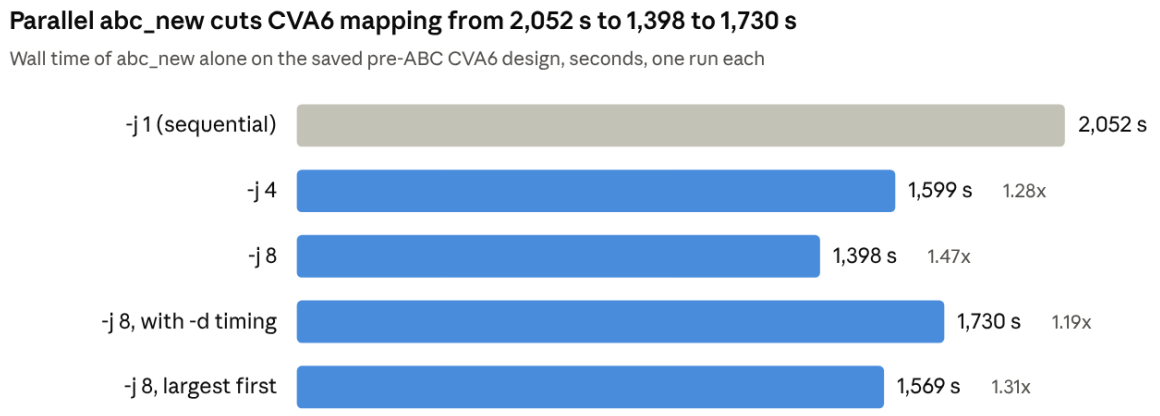


Figure 2. Wall time of abc_new alone on the saved pre-ABC CVA6 design, one run each.

Run	abc_new wall time (s)	Speedup	Peak memory (MB)
-j 1 (sequential)	2,052	1.00×	2,051
-j 4	1,599	1.28×	1,614
-j 8	1,398	1.47×	1,619
-j 8, with -d timing	1,730	1.19×	1,609
-j 8, largest first (final build)	1,569	1.31×	1,607

7.3 Where the time goes

Yosys per-pass timers report process CPU time, including reaped yosys-abc children. In the -j 8 run with -d, abc_new took 1,962 CPU-seconds against 1,730 s of wall time:

Part of abc_new	Calls	CPU time (s)	Runs on
ABC processes (sum of ABC's elapsed times)	93	869	worker threads
write_xaiger2	186	392	main thread
portarcs	76	296	main thread
clean, abc9_ops, abc_ops_reintegrate, read_aiger	545	56	main thread

- **Serial floor:** about 740 s of main-thread work cannot overlap anything; most of it is write_xaiger2 and portarcs normalizing the whole 93-module design, about 4 s per call.
- **ABC is not the bulk:** in the sequential run ABC's elapsed times sum to 679 s of 2,052 s; under parallel load they rise to 869 s.
- **Critical path:** hpdcache_wbuf is the longest single ABC run in both modes (154 s sequential, 363 s under load); its parents up to cva6 wait for it.

8. Issues found and fixes

CVA6 exposed two issues that the regression tests missed; both are fixed, in 48001b2d4 and db1edb158.

8.1 Box-id assertion (crash)

- **Symptom:** the first full ORFS run died at 28 min 44 s: Assert mapped_cell->type == "$__boxid %d" failed in abc_ops_reintegrate.cc:450.
- **Cause:** collecting a deferred box runs abc9_ops -prep_box, which renumbers abc9_box_id on every box module. A module prepared before that but collected after it gets $__boxidN names that no longer match.
- **Why it was not seen before:** the sequential loop prepares and collects each module back to back, so no renumbering can land in between.

- **Fix:** commit 48001b2d4: prepare snapshots every abc9_box_id; collect applies the snapshot around read_aiger and reintegrate, then restores the current ids. Reintegrate finds boxes by cell name, so the id is only a consistency check.

8.2 Leftover $input_port cells

- **Symptom:** parallel output had 74,100 $input_port cells against 1,300 sequential, inflating the stat cell count from 210,425 to 247,497 while real cells were identical.
- **Cause:** portarcs switches the whole design into buffer-normalized mode and back but leaves its $input_port cells; write_xaiger2 removes them, so sequentially the next prepare cleans up, but in parallel many collects follow the last prepare.
- **Why it was not seen before:** the regression tests check equivalence, not stray bookkeeping cells.
- **Fix:** commit db1edb158: map_parallel() removes all $input_port cells once at the end; CVA6 now has 0.

9. Repositories and publishing

Both branches are pushed to Pr0Chef forks on GitHub and not yet merged. The KUL submodule pins 7b7e811cb, which exists in the Pr0Chef/yosys fork.

Checkout	Branch	Remotes	Head
~/Desktop/projects/exodus/yosys (build tree)	abc-new-parallel	origin = YosysHQ/yosys	7b7e811cb
~/Desktop/projects/KUL	abc-new-parallel	origin = Pr0Chef/KUL	1a7fd02
~/Desktop/projects/KUL/yosys (submodule)	abc-new-parallel	origin = Pr0Chef/yosys (pushes over SSH), upstream = YosysHQ/yosys	7b7e811cb

9.1 Published

Repository	Branch	Commits over main	Head
github.com/Pr0Chef/yosys (fork of YosysHQ/yosys)	abc-new-parallel	6	7b7e811cb
github.com/Pr0Chef/KUL	abc-new-parallel	2	1a7fd02

All eight commits are authored by rafay. To get the change in a fresh clone:

```
git clone -b abc-new-parallel git@github.com:Pr0Chef/KUL.git
cd KUL && git submodule update --init --recursive
```

9.2 To reproduce

```
cmake -S yosys -B yosys/build && cmake --build yosys/build -j8
export YOSYS_EXE=$PWD/yosys/build/yosys OPENROAD_EXE=$(command -v openroad)
./run_cva6_asap7.sh -s synth -H -j 8
```

yosys-abc must sit next to the yosys binary, or be named with -exe.

10. Limitations and next steps

The most valuable next step is the per-module buffer normalization in section 11, which removes most of the ~740 s serial floor.

10.1 Limitations

- **Serial bookkeeping:** write_xaiger2 and portarcs normalize the whole design per module: 262 calls at about 4 s each on CVA6.
- **Collect in fixed order:** a finished module waits for every module before it in topological order, so a parent can start slightly later than needed.
- **Disk:** every ready module's ABC input is written up front; all 93 temp dirs can exist at once.
- **Output names:** -j 1 and parallel runs generate different net and cell names; the mapped logic is the same.
- **Platforms:** Windows, WebAssembly and linked-ABC builds always use the sequential loop.

10.2 Next steps

- ☐ Add a regression test that maps a deferred box alongside a module containing another box.
- ☐ Run full ORFS CVA6 hierarchical synthesis end to end with the final build and compare netlist and timing with the baseline.
- ☐ Run each -j setting several times to put error bars on the wall-time numbers.
- ☐ Consider proposing the change upstream to YosysHQ; abc_new is still marked experimental there.

11. Future work

Planned but not started.

11.1 Per-module buffer normalization

- **Problem:** write_xaiger2 (186 calls, 392 s) and portarcs (76 calls, 296 s) call design->bufNormalize(true) and then (false), rewriting all 93 modules each time though each call needs one. This serial work caps -j 8 at 1.2 to 1.5 times, and sequential abc_new pays it too.
- **Proposed change:** confirm module-scoped normalization (module->bufNormalize()) is safe; normalize only the target module and the box modules it instantiates in write_xaiger2, and only the target module in portarcs; remove $input_port cells only there; re-verify with the 44 tests and the CVA6 A/B.

- **Expected gain:** if the ~740 s drops to a few seconds, abc_new on CVA6 could fall to a few hundred seconds, bounded by the longest dependent ABC chain led by hpdcache_wbuf (150 to 360 s). An estimate that depends on the first step.
- **Risks:** buffer-normalized mode is a design-level flag other code may rely on; both passes are used by other flows, so testing must go beyond the abc_new suite.